Emergency Training Simulator POC

AI-driven adaptive emergency-response training simulator

Proof-of-concept project update

First audience

This update is framed for Dr. Cameron.

The goal is to show:

- what I built
- what it proves
- what it does not prove yet
- why the build order is careful
- what the next useful step is

This is more on-campus project update than startup pitch.

The short version

I am building a small backend-first proof of concept for emergency-response training.

The full project goal is a real-time adaptive training platform with AI, immersive environments, sensing, validation, and scalable deployment.

The current repo is the first foundation step: prove that an action can move through the backend, update state, get checked, get logged, and be reviewed later.

Full-system north star

The larger project eventually points toward:

- agentic AI scenario generation
- real-time scenario adaptation
- immersive rooms and mannequins
- multimodal sensing
- structured verification and validation
- hub-and-spoke deployment
- instructor review and continuous improvement

This POC does not build all of that yet. It builds the first trusted loop underneath it.

Why this project exists

Emergency training is hard to scale because good practice needs:

- clear scenarios
- visible consequences
- instructor review
- repeatable sessions
- room for environmental cues

This POC starts with the smallest useful version of that loop.

The problem I am avoiding

It would be easy to jump straight into Unreal, AI agents, 3D assets, sensors, and physics.

That would look cooler early, but it would hide the hardest question:

Can the system keep a clean, trusted record of what happened?

So the backend comes first.

How I started

Sprint 0 was repo bootstrapping.

I created:

- project docs
- architecture notes
- message schemas
- scenario files
- validation rules
- backend placeholders
- Unreal placeholder docs
- asset pipeline placeholders

The goal was to set the rails before writing real behavior.

The POC loop

Emergency Training Simulator POC / backend-first local proof

The main loop is:

```
manual or stubbed trainee action
-> backend receives event
-> scenario engine updates state
-> validation rules check it
-> client receives state
-> JSONL log is written
-> replay and review summarize the session
```

That is the heartbeat of the whole project.

Where I am now

Emergency Training Simulator POC / backend-first local proof

The repo now has a working backend slice.

It includes:

- FastAPI health endpoint
- WebSocket session endpoint
- deterministic scenario engine
- POC validation rules
- basic POC security guardrails
- JSONL session logs
- replay and review scripts
- one-command local demo
- future Unreal client contract
- text-only Unreal display preview

Current scenario

The first scenario is a tiny smoke-inhalation room POC.

It is intentionally small.

The happy path is:

1. assess the scene
2. confirm PPE
3. request backup
4. move the wooden chair
5. open the entry door
6. enter the room
7. remove the occupant

This is not real emergency guidance. It is a software test scenario.

Environment cues

The scenario now includes two environment objects:

- ‘wooden_chair_poc‘
- ‘entry_door_poc‘

The chair can block the path.

The door can be open or closed.

The backend tracks these as state, not as 3D gameplay objects.

Why the chair and door matter

These objects prove a useful idea:

The simulator should not ignore the room.

If a trainee rushes past a blocked path or closed door, the system can warn them.

Right now that warning is simple rule-checking.

Later, richer object detail can support collision, mass, force direction, gravity notes, and Unreal display behavior.

Validation boundary

Emergency Training Simulator POC / backend-first local proof

Validation in this repo means:

"Did the action follow this toy scenario’s rules?"

It does not mean:

- medical correctness
- firefighter protocol authority
- EMT certification guidance
- real-world incident command advice

This line matters because the POC should be useful without pretending to be official.

Security baseline

The client is not trusted as the source of truth.

The client can say:

"I want to attempt this action."

The backend decides:

- what state changed
- whether the action was accepted
- whether warnings or errors exist
- whether the scenario ended
- what gets logged for review

Current guardrails

The backend now rejects:

- unexpected message fields
- weird or path-like IDs
- unsafe scenario IDs
- unsafe action names
- oversized or deeply nested payloads
- wrong sequence numbers
- duplicate event IDs

This is not production security yet. It is the first lock on the first door.

Sample demo result: happy path

The happy path proves the full loop.

Expected review:

- final state: occupant removed
- chair moved: yes
- door opened: yes
- warnings: none
- errors: none
- end condition reached: yes
- replay verification: passed

This shows the backend can accept good ordered actions and produce a clean review.

Sample demo result: warning/error path

The warning path proves the system can catch bad order or ignored cues.

Example review:

- trainee tries to enter too early
- chair or door cue is still unresolved
- validation returns warnings or errors
- state does not advance incorrectly
- log still records what was attempted
- replay can explain what happened later

That review trail is one of the most important parts of the POC.

Sample review output

Emergency Training Simulator POC / backend-first local proof

Audience context: this is the proof point behind the demo.

```
    Post-scenario review
    Session: warning/error path
    Final phase: stopped
    Actions reviewed: 8
    Validation counts: ok=5, warning=2, error=1
    Replay verified: yes

    Action timeline:
    1. confirm_ppe -> warning
    2. enter_room -> error
    7. enter_room -> warning

    Key observations:
    - The log captured 8 trainee actions in order.
    - The entry door ended closed or blocking the path.
```

The point is not "the demo ran."

The point is "the system can explain what happened."

Unreal client contract

Unreal is treated as a future client, not the source of truth.

For now, the repo defines:

- WebSocket lifecycle
- example session messages
- client-side state machine
- text-only display contract
- Python harness that behaves like a future Unreal client

No Unreal scene is built yet.

Asset pipeline direction

Future assets may come from tools like Hunyuan 3D, Blender, or similar workflows.

But the repo does not generate assets yet.

The planned pipeline is:

```
candidate asset
-> manual Blender review
-> metadata check
-> approved/reviewed asset folder
-> Unreal import later
```

The current chair and door are metadata and scenario objects only.

Next goal

The next useful goal is a thin display-side demo contract.

That means:

- keep backend authority
- keep scenario logic in Python
- let a future client display state clearly
- show environment cues in a simple text debug panel
- avoid 3D and physics until the data contract is stable

Basically: make the future Unreal side boring first, then make it visual later.

How I get there

Near-term steps:

1. keep improving the backend contract
2. keep tightening the security boundary
3. add richer environment object fields carefully
4. keep replay and review as the proof point
5. build tiny Unreal display tests only when needed
6. review generated assets manually before using them
7. add physics only after object state is clean

This keeps the project from turning into soup.

Ideal end result

The ideal system is an adaptive training simulator where:

- trainees act inside a scenario
- the room state changes
- environmental cues matter
- instructors can review what happened
- AI helps generate or adapt safe training content
- Unreal displays the state
- logs support learning and debriefing

That is the long-term picture, not what I claim to have today.

What this is not

This is not:

- a certified training system
- a medical decision tool
- a firefighter protocol engine
- a real physics simulator
- a production Unreal build
- a sensor platform
- a cloud deployment
- a production-secure app

It is a strong first loop.

And that is exactly what it should be right now.

Closing

The project is moving in the right order:

First, prove the loop.

Then, prove replay and review.

Then, prove client display.

Then, add richer objects.

Then, when the foundation is stable, start wrestling with Unreal, Blender, generated assets, and physics.

One monster at a time.

Best 10-minute demo flow

For Dr. Cameron:

- 5 minutes: explain the loop, scope, and trust boundary
- 5 minutes: run the local demo and point at replay/review

The core takeaway should be:

The backend can keep track of what happened, explain it afterward, and leave room for Unreal/UX/physics later.